Structuri de Date
Laboratorul 5: Arbori binari

Mihai Nan

22 martie 2026

1. Introducere

Un arbore combină avantajele oferite de alte două structuri: tablourile și listele înlănțuite. Arborii permit, pe de o parte, executarea unor căutări rapide, la fel ca tablourile ordonate, iar, pe de altă parte, inserarea și ștergerea elementelor sunt rapide, la fel ca la o listă simplu înlănțuită.

Dacă ne-am uitat la un arbore genealogic, sau la o ierarhie de comandă într-o firmă, am observat informațiile aranjate într-un arbore. Un arbore este compus dintr-o colecție de **noduri**, care sunt unite prin **arce**, unde fiecare nod are asociată o anumită informație, și o colecție de copii. Vom reprezenta nodurile prin cercuri, iar arcele, prin linii care unesc cercurile. **Copiii** unui nod sunt acele noduri care urmează imediat sub nodul respectiv. **Părintele** unui nod este acel nod care se află imediat deasupra. **Rădăcina** unui arbore este acel nod unic, care nu are niciun părinte.

Nodurile reprezintă, de regulă, entități din lumea reală, cum ar fi valori numerice, persoane, părți ale unei mașini, rezervări de bilete de avion. Nodurile sunt elemente obișnuite pe care le putem memora în orice altă structură de date. Arcele dintre noduri reprezintă modul în care nodurile sunt conectate. Este ușor și rapid să ajungem de la un nod la altul dacă acestea sunt conectate printr- un arc. Arcele sunt reprezentate în C prin **pointeri**.

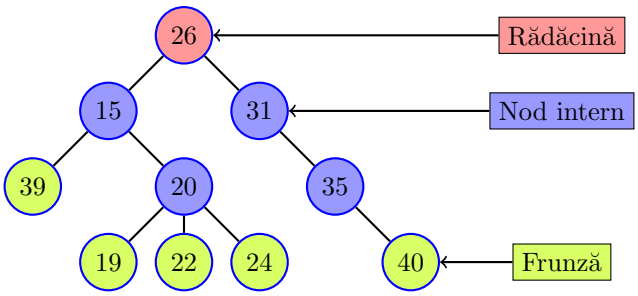


Figura 1: Un exemplu de arbore

Toți arborii au următoarele proprietăți:

- Există o singură rădăcină.

- Toate nodurile, cu excepția rădăcinii, au exact un părinte.
- Nu exista cicluri. Cu alte cuvinte, pornind de la un anumit nod, nu există un anumit traseu pe care îl putem parcurge astfel încât să ajungem înapoi la nodul de plecare.

Înălțimea unui arbore este, de fapt, valoarea maximă de pe nivelurile nodurilor terminale. Numărul de descendenți direcți ai unui nod reprezintă **ordinul** sau **gradul nodului**. **Ordinul** sau **gradul arborelui** este valoarea maximă luată de gradul unui nod component al arborelui.

2. Arborele binar

Arborii binari sunt un tip aparte de arbori, în care fiecare nod are maxim 2 copii. Pentru un nod dat, într-un arbore binar, vom avea fiul din stânga și fiul din dreapta. Deci, un nod dintr-un arbore binar poate avea 2 copii (stânga și dreapta), un singur copil (doar stânga sau doar dreapta) sau niciun copil. Nodurile care nu au niciun copil se numesc **noduri frunză**, iar cele care au 1 sau 2 copii se numesc **noduri interne**.

Între numărul **N** de noduri al unui arbore binar și înălțimea sa **H** există relațiile:

$$H \leq N \leq 2^H - 1 \tag{1}$$

$$\log_2 N \leq H \leq N \tag{2}$$

2.1. Arborele binar de căutare

Într-un **arbore binar de căutare**, pentru fiecare nod cheia acestuia are o valoare mai mare decât cheile tuturor nodurilor din sub-arborele stâng și mai mică decât cheile nodurilor ce compun sub-arborele drept. Arborii binari de căutare permit menținerea datelor în ordine și o căutare rapidă a unei valori, ceea ce îi recomandă pentru implementarea de mulțimi și dicționare ordonate.

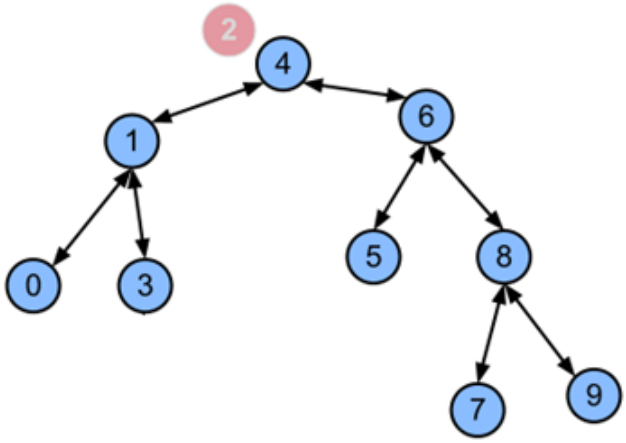
Observație

O importantă proprietate a arborilor de căutare este aceea că **parcurgerea în inordine** produce o secvență ordonată crescător a cheilor din nodurile arborelui.
Valoarea maximă dintr-un arbore binar de căutare se află în nodul din **extremitatea dreapta** și se determina prin coborârea pe sub-arborele drept, iar **valoarea minimă** se află în nodul din **extremitatea stânga**.

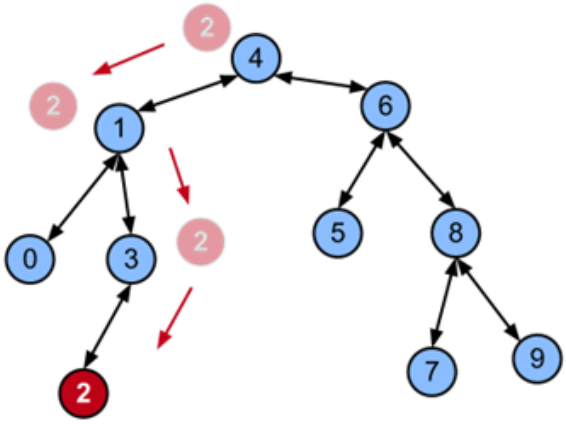
Pentru reținerea informației, vom folosi alocarea dinamică. Pentru fiecare nod în parte este necesar să se rețină, pe lângă informația utila, și legăturile cu nodurile copii (adresele acestora), iar pentru aceasta uzităm pointeri. În definirea de mai jos, ***left*** reprezintă adresa nodului copil din stânga, ***value*** reprezinta câmpul cu informație utilă, iar ***right*** este legătura cu copilul din dreapta.

```c
typedef struct node {
    Item value;
    struct node *left;
    struct node *right;
} TreeNode, *Tree;
```

2.1.1. Inserarea unui nod



Să considerăm arborele de mai sus și să presupunem că dorim să inserăm nodul cu valoarea 2. Acesta se va insera ca nod frunză. Pentru a-l insera, va trebui să căutăm o poziție în arbore care respectă regula de integritate a arborilor binari de căutare.



Vom începe prin compararea nodului de inserat (2) cu rădăcina arborelui dat (4). Observăm că este mai mic decât ea, deci va trebui inserat undeva în sub-arborele stâng al acesteia. Vom compara apoi 2 cu 1. Din moment ce 2 este mai mare decât 1, nodul cu valoarea 2 va trebui plasat undeva în sub-arborele drept al lui 1. Se compară apoi 2 cu 3. Deoarece 3 este mai mare decât 2, nodul cu valoarea 2 trebuie să se afle în sub-arborele din stânga al nodului 3. Dar nodul 3 nu are niciun copil în partea stângă. Asta înseamnă că am găsit locația pentru nodul 2. Tot ceea ce mai trebuie făcut este să modificăm în nodul 3 adresa către fiul sau stâng astfel încât să indice spre 2. (Vezi fig. de mai sus!)

Algorithm 1 Insert

```
procedure INSERT(root, value)
    if root = NULL then
        initTree(root, value)
        return root
    else if root → value = value then
        print(Nodul exista)
        return root
    else
        if root → value > value then
            if root → left = NULL then
                initTree(root → left, value)
                return root
            else
                root → left ← insert(root → left, value)
                return root
        else
            if root → right = NULL then
                initTree(root → right, value)
                return root
            else
                root → right ← insert(root → right, value)
                return root
```

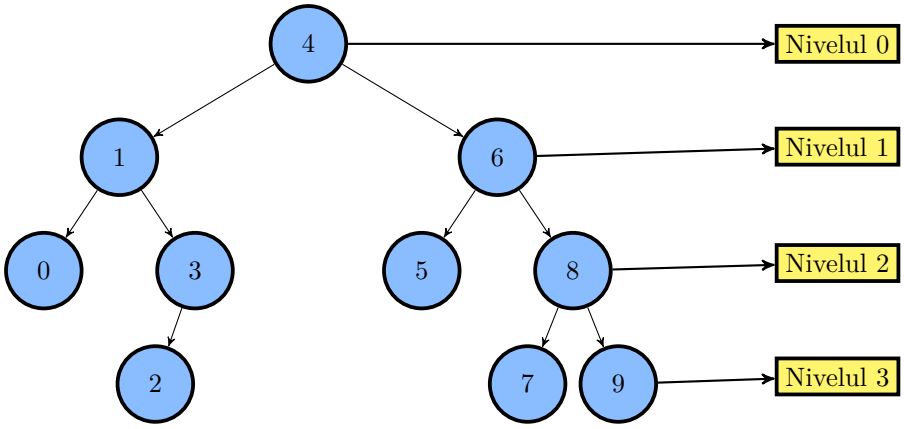
2.1.2. Înălțimea arborelui

Înălțimea unui arbore reprezintă nivelul maxim din arbore (adâncimea frunzei de nivel maxim).

Înălțimea unui arbore binar cu N noduri interne este minim $\log_2 N$ şi maxim $N - 1$.

Înălțimea unui arbore binar complet cu M noduri este cel mult $O(\log M)$.

Pentru a determina cum se calculează înălțimea unui arbore binar, pornim de la următorul exemplu:

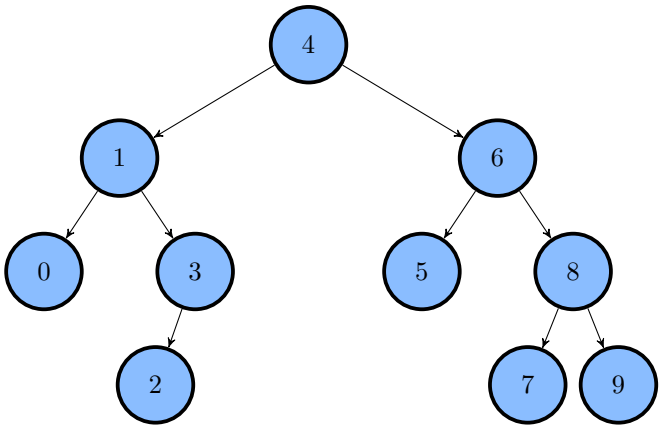


Algorithm 2 Height

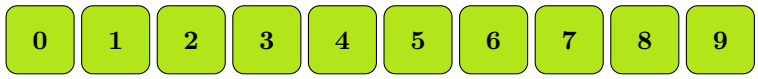
```
procedure HEIGHT(root)
    if root = NULL then
        return 0
    else
        tmp ← root
        left ← height(tmp → left)
        right ← height(tmp → right)
        if left ≥ right then
            left ← left + 1
            return left
        else
            right ← right + 1
            return right
```

2.1.3. Parcurgerea arborelui

Există 3 tipuri de parcurgere a unui arbore: **inordine**, **preordine** și **postordine**. Aceste denumiri corespund modului cum este vizitată rădăcina.



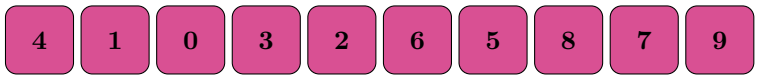
Parcurgerea în inordine



Algorithm 3 Inordine

```
procedure INORDINE(root)
    if root ≠ NULL then
        inordine(root → left)
        print(root → value)
        inordine(root → right)
```

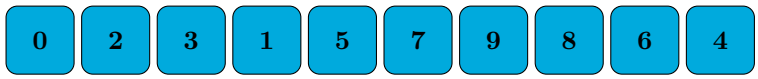
Parcurgerea în preordine



Algorithm 4 Preordine

```
procedure PREORDINE(root)
    if root ≠ NULL then
        print(root → value)
        preordine(root → left)
        preordine(root → right)
```

Parcurgerea în postordine



Algorithm 5 Postordine

```
procedure POSTORDINE(root)
    if root ≠ NULL then
        postordine(root → left)
        postordine(root → right)
        print(root → value)
```

2. Cerințe

În acest laborator dispuneți de mai multe fișiere, inclusiv scheletul de cod, după cum urmează:

- `tree.h` – fișierul în care este definită structura de date cu care vom lucra și antetele funcțiilor care trebuie implementate.
- `tree.c` – fișierul în care se vor implementa funcțiile ce rezolvă cerințele laboratorului.
- `main.c` – checker pentru validarea implementării cerințelor laboratorului.
- `Makefile` – fișierul pe baza căruia se vor compila și rula testele.

Pentru compilarea tuturor aplicațiilor, folosiți comanda "`make build`". Aceasta are următorul output pentru un program fără erori de sintaxă sau warning-uri:

```
$ make build
gcc -c tree.c -g
gcc main.c tree.o -o testTree -g -Wall
```

Iar pentru ștergerea automată a fișierelor generate prin compilare folosiți comanda "`make clean`":

```
$ make clean
rm -f *.o *~ testTree *.dot *.png
```

Pentru testarea completă (inclusiv memory leaks) puteți folosi "`make test`".

Cerința 1 (6 puncte) Implementați următoarele funcții de lucru cu arbori binari în fișierul "`tree.c`".

a) `void init(Tree *root, Item value)` – Funcție care inițializează un nod de arbore: îi alocă memorie, îi setează câmpul valoare, setează left și right să pointeze către `NULL`.

b) `Tree insert(Tree root, Item value)` – Funcție care inserează o valoare într-un arbore binar, respectând proprietățile unui arbore binar de căutare. Funcția va returna arborele nou creat.

> **Atenție**
>
> Funcția poate primi ca parametru arborele vid (`NULL`) și va trebui să returneze un arbore format dintr-un singur nod.

c) `void printPostorder(Tree root)` – Funcție care afișează nodurile folosind parcurgerea în postordine.

d) `void printPreorder(Tree root)` – Funcție care afișează nodurile folosind parcurgerea în preordine.

e) `void printInorder(Tree root)` – Funcție care afișează nodurile folosind parcurgerea în inordine.

f) `void freeTree(Tree *root)` – Funcție care dealocă întreaga memorie alocată pentru un arbore binar. `root` va pointa către `NULL` după ce se va apela funcția.

Atenție

Punctajul pentru `freeTree` (0.75 puncte) se acordă automat doar dacă `valgrind` (Linux) sau `leaks` (macOS) confirmă că nu există memory leaks.

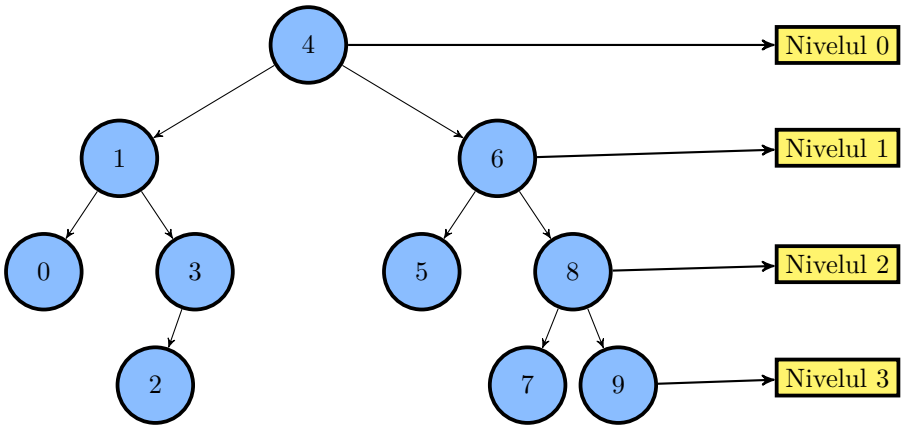
g) `int size(Tree root)` – Funcție care determină numărul de noduri dintr-un arbore binar.

h) `int maxDepth(Tree root)` – Funcție care returnează adâncimea maximă a arborelui.

Atenție

Nivelul sau **adâncimea** unui nod reprezintă numărul de arce de la rădăcină la nod (rădăcina are nivel 0).

Exemplu: Adâncimea maximă a arborelui de mai jos este 3.

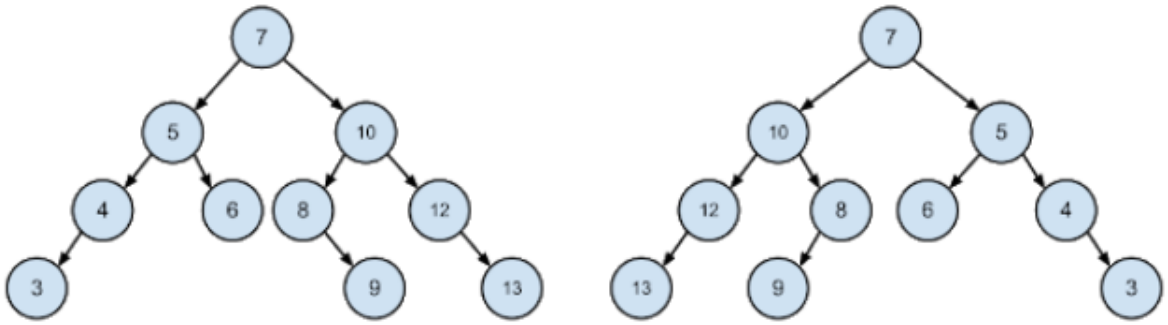


Cerința 2 (2 puncte) Implementați, în fișierul `tree.c`, o funcție care construiește oglinditul unui arbore.

```
void mirror(Tree root)
```

Observație

Funcția va aplica modificările direct pe arborele primit ca parametru prin pointer la nodul rădăcină.



Cerința 3 (2 puncte) Implementați, în fișierul `tree.c`, o funcție care verifică dacă doi arbori binari sunt identici.

`int sameTree(Tree root1, Tree root2)` – funcția returnează 1 dacă arborii sunt identici și 0 altfel.

3. Informații utile

Observații:

- Punctajul pentru `freeTree` (0.75 puncte) se acordă automat prin `make test`, condiționat de rezultatul `valgrind` (Linux) sau `leaks` (macOS).
- Parcurgerile (`printPostorder`, `printPreorder`, `printInorder`) sunt verificate **automat** – checker-ul captează output-ul funcțiilor și îl compară cu rezultatul așteptat.
- Dacă un test pică, checker-ul afișează **doar primul eșec** și continuă cu următoarea funcție.
- Checker-ul rulează **toate testele** până la final, chiar dacă unele funcții pică.

Se pot rula și teste individuale folosind `./testTree <nr>`:

```
./testTree <nr1> [<nr2> ...]
```

unde numerele corespund funcțiilor:

Nr.	Funcție	Puncte
1	insert + init	1.50
2	printPostorder	0.75
3	printPreorder	0.75
4	printInorder	0.75
5	size	0.75
6	maxDepth	0.75
7	mirror	2.00
8	sameTree	2.00
freeTree (valgrind/leaks)		0.75
Total		**10.00**

Exemple de testări:

- Testarea unei singure funcții:

```
$ ./testTree 1
insert  - Toate testele au trecut!  Puncte: 1.50
```

- Testarea mai multor funcții simultan:

```
$ ./testTree 1 5 6
insert    - Toate testele au trecut!  Puncte: 1.50
size      - Toate testele au trecut!  Puncte: 0.75
maxDepth  - Toate testele au trecut!  Puncte: 0.75
```

- Testarea doar a parcurgerilor:

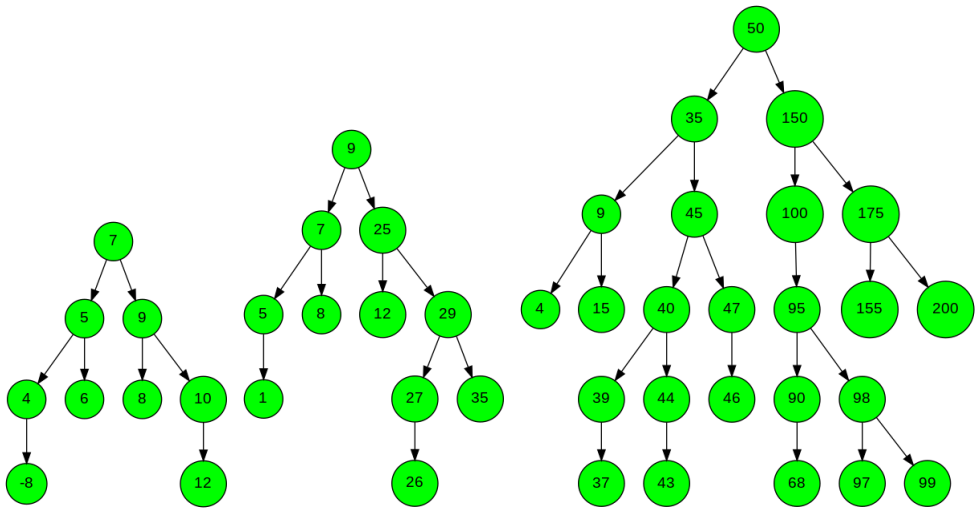
```
$ ./testTree 2 3 4
postorder - Toate testele au trecut!  Puncte: 0.75
preorder  - Toate testele au trecut!  Puncte: 0.75
inorder   - Toate testele au trecut!  Puncte: 0.75
```

- Dacă `graphviz` este instalat, checker-ul generează automat imagini PNG cu arborii construiți:
  - `tree1.png`, `tree2.png`, `tree3.png` – arborii BST (verde)
  - `mirror1.png`, `mirror2.png`, `mirror3.png` – arborii oglindiți (roșu)

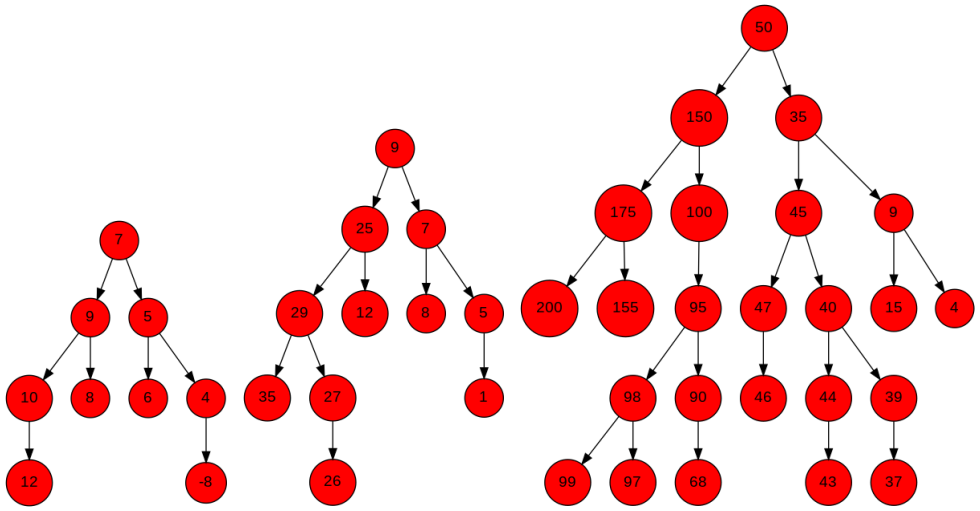
  Pentru a instala `graphviz`:

```
sudo apt install graphviz        # Linux
brew install graphviz             # macOS
```

  Exemple de imagini generate:



Arborii rezultați în urma aplicării operației de oglindire sunt următorii:



- Testarea tuturor funcțiilor (implementare corectă):

```
$ make test
insert      - Toate testele au trecut!  Puncte: 1.50  (vezi tree1/2/3.png)
postorder   - Toate testele au trecut!  Puncte: 0.75
preorder    - Toate testele au trecut!  Puncte: 0.75
inorder     - Toate testele au trecut!  Puncte: 0.75
size        - Toate testele au trecut!  Puncte: 0.75
maxDepth    - Toate testele au trecut!  Puncte: 0.75
mirror      - Toate testele au trecut!  Puncte: 2.00  (vezi mirror1/2/3.png)
sameTree    - Toate testele au trecut!  Puncte: 2.00
========================================
Verificare Valgrind (freeTree):
  PASSED - Fara memory leaks!
freeTree                Puncte: 0.75
========================================
Toate testele au trecut! Felicitari!
Punctajul obtinut este: 10.00
```

- Exemplu cu o funcție implementată greșit (`inorder` fără parcurgere pe dreapta):

```
$ make test
insert      - Toate testele au trecut!  Puncte: 1.50
postorder   - Toate testele au trecut!  Puncte: 0.75
preorder    - Toate testele au trecut!  Puncte: 0.75

  inorder   test1 PICAT
  Input (ordinea inserarii): [7, 5, 9, 4, 10, 6, 8, -8, 12]
  Asteptat: [-8, 4, 5, 6, 7, 8, 9, 10, 12]
  Primit:   [-8, 4, 5, 7]
inorder     - PICAT
size        - Toate testele au trecut!  Puncte: 0.75
maxDepth    - Toate testele au trecut!  Puncte: 0.75
mirror      - Toate testele au trecut!  Puncte: 2.00
sameTree    - Toate testele au trecut!  Puncte: 2.00
========================================
Verificare Valgrind (freeTree):
  PASSED - Fara memory leaks!
freeTree                Puncte: 0.75
========================================
Punctajul obtinut este: 9.25
```

- Exemplu cu memory leaks (`freeTree` nu eliberează toată memoria):

```
$ make test
...
========================================
Verificare Valgrind (freeTree):
  FAILED - Memory leaks detectate!
   LEAK SUMMARY:
      definitely lost: 2,016 bytes in 84 blocks
      indirectly lost: 6,840 bytes in 285 blocks
freeTree                Puncte: 0.00
========================================
Punctajul obtinut este: 9.25
```